

OEIS Program Synthesis, Verified Semantics, and Graded Similarity: A Unified Account

Zarathustra J. Goertzel

July 19, 2026

Abstract

We collect an ongoing line of work built around the On-Line Encyclopedia of Integer Sequences (OEIS) as a testbed for verified program synthesis. Three strands meet here. First, a small domain-specific language (DSL) of integer-sequence programs is given a machine-checked operational semantics in Lean, together with a token *recognizer*, a big-step generative-semantics (GSLT) layer with modal emission specifications, and a family of formal *obligations* that a synthesizer must respect. Second, a neural sequence-to-graph synthesizer is organized so that the network’s operational contracts—a legality mask, a canonical form, relabeling invariance—are exactly the theorems the prover discharges; the architecture is read as a specification. Third, a graded similarity layer expressed in probabilistic-logic truth values $\langle \text{strength}, \text{confidence} \rangle$ recovers the structure that an exact deduplication hash discards: it audits train/test leakage and, via a *sound abstract interpreter*, attaches provable properties (totality, sign, parity) to each program. We report concrete measurements over a corpus of 104,083 synthesized programs, and we argue that the same provable-property machinery serves two roles in an OEIS synthesis loop: it *filters* candidate programs and it *measures* them theoretically, turning a leakage audit into a verified conjecture generator.

1 Introduction

The OEIS is a natural benchmark for program synthesis: each entry is an integer sequence, and a solution is a program that reproduces its terms. Recent self-learning systems synthesize such programs at scale from a small combinator DSL, discovering programs for a large fraction of the OEIS without human-written training data [1, 2]. Our contribution is not another synthesizer but a *verified and measured* account of the objects a synthesizer manipulates.

We separate three concerns that are often conflated:

- **What a program *is*** — its syntax over a fixed operator vocabulary, and the machine-checked semantics of running it (Sections 2–3).
- **What a program *does*** — its input/output behaviour on a finite probe window (its *signature* σ), the extensional object (Section 5).
- **What a program *provably has* as properties** — totality, sign, parity, growth: the intensional object, obtained by sound static analysis rather than read off finitely many outputs (Section 6).

operator	arity	role	partiality / termination
zero,one,two	0	constants	total
x,y	0	variables ($n \geq 0$; inner ≥ 0)	total
addi,diff,mult	2	arithmetic	total (resource: overflow)
divi,modu	2	division / modulo	<i>genuine</i> : error on zero divisor
cond	3	branch on ≤ 0	total given branches
loop,loop2	3,5	bounded iteration	terminating (resource: timeout)
compr	2	unbounded search	<i>genuine</i> : may not terminate
push	2	stack push	total (resource: push limit)
pop	1	stack pop	<i>genuine</i> : error on empty

Table 1: The synthesis DSL. Only **divi/modu** (zero divisor), **pop** (empty stack), and **compr** (non-terminating search) introduce *genuine* partiality; **loop/loop2**, overflow, and timeouts are *resource* limits. This three-way character drives the totality analysis.

The payoff of keeping these apart is a clean division of labour: the extensional layer gives cheap graded evidence but can never certify behaviour beyond the probe window; the intensional layer certifies properties for all inputs but is conservative. Combined, they sharpen both benchmark hygiene and the discovery of conjectures.

2 The OEIS synthesis DSL

Programs are postfix token streams over a 16-operator vocabulary evaluated on a two-register integer state. The operators are constants **zero**, **one**, **two**; the index variable **x** = n and an inner variable **y**; binary arithmetic **addi**, **diff**, **mult**, **divi**, **modu**; a conditional **cond** (test ≤ 0); bounded iterators **loop** and **loop2**; an unbounded search **compr** (the a -th $n \geq 0$ satisfying a predicate); and stack operators **push**, **pop**. Table 1 summarizes the arities and the partiality character of each operator, which is decisive for the analysis of Section 6.

A closed form illustrates the language: the factorial $n!$ is **loop**(**mult**(**x**,**y**), **x**, **one**) — fold multiplication by the running counter n times starting from 1. The counting sequence $a(n) = n + 1$ is **addi**(**x**,**one**).

3 Verified GSLT semantics

The DSL is given a machine-checked operational semantics in Lean. On top of it we build a generative-semantics layer as a triple (T, E, R) : terms T , an equational theory E (a setoid), and a rewrite relation R with left/right coherence laws. The coarse *big-step GSLT* rewrites a program to the integer value its fuel-bounded evaluator emits; a completed-value term is never reflexively a program, so non-triviality is load-bearing. Two modalities express synthesis goals:

$$\begin{aligned} \Diamond(\text{emits } v \text{ at } k) &\iff \exists s', \text{eval fuel sig } p(\text{seed } k) = \text{some}(v, s'), \\ \text{EmitsPrefix } p \ell &\iff \forall i v, \ell_i = v \Rightarrow \Diamond(\text{emits } v \text{ at } i). \end{aligned}$$

EmitsPrefix is precisely the OEIS synthesis specification: “ p reproduces the observed prefix ℓ .”

Obligations. A synthesizer that claims correctness must respect a family of formal obligations. We single out three that are discharged as machine-checked theorems, axiom-clean (each theorem’s `#print axioms` footprint lies within `{proptext, Quot.sound}`):

Obligation 1 (Recognizer completeness / mask equivalence). *The legality predicate `maskLegal` on token streams coincides with the executable recognizer (`recognize_iff_maskLegal`), and every well-formed program is recognized (`recognize_complete`). Hence a constrained decoder that emits only mask-legal tokens can generate exactly the well-formed programs.*

Obligation 2 (Relabeling invariance). *Evaluation is invariant under a signature isomorphism (`eval_relabel_signature_iso`): renaming primitives consistently does not change the computed sequence. Behaviour depends on structure, not on operator names.*

Obligation 3 (Canonicalization soundness). *A flag-driven canonical form (sorting the children of commutative operators, etc.) is semantics-preserving (`canonicalize_sound`, `orgCanonicalize_sound`), so deduplicating by canonical form never merges behaviourally distinct programs.*

Further obligations (equational-theory soundness, eval-congruence for the modalities, finite-probe rigidity, small-step/big-step agreement) organize the remaining work. Finite-probe rigidity—whether agreement on a finite probe entails equality—is the formal shadow of the leakage phenomenon in Section 5.

4 The synthesizer as specification

The neural synthesizer maps a sequence prefix to a program graph (sequence-to-graph). Its decoder is *execution-guided*: at each step a legality mask restricts the token set to those that can extend a well-formed program, and the mask is exactly the recognizer of Obligation 1. The training ladder A0–A3 rises from a plain conditioned decoder (A0) to retrieval-augmented and contrastive variants (A3) that consult a similarity index. The central design stance is that the network’s operational contracts are the prover’s theorems: the mask’s soundness, the canonical form’s soundness, and relabeling invariance are not engineering conveniences but the denotational safety envelope of the synthesizer.

5 Leakage-safe data and the extensional layer

Exact deduplication and splits. Each program’s probe signature σ is its output vector on $n = 0, \dots, 31$ at a fixed fuel, with $\perp$ (`None`) marking a timeout. Programs with identical σ form an exact behavioural equivalence class (“E-class”); the train/val/test split is assigned by whole connected component so that no exact behavioural duplicate straddles the split boundary. Over 104,083 programs this gate is leak-tight for exact duplicates: all 3,007 exact- σ duplicate pairs lie in the same split.

Graded similarity. Exact equality is brittle. We express graded similarity as a probabilistic-logic truth value $\langle s, c \rangle$ [6]: the extensional strength is behavioural agreement over the *defined* domain, $s = |\text{agree}|/|\text{informative}|$, and confidence follows the evidence-count law $c = m/(m + k)$ with m the number of informative probe positions. Exact equality is the degenerate case $s = 1$, $m = 32$. A locality-sensitive index over signatures generates near-equivalent candidate pairs without an $O(n^2)$ scan.

strength band	cross-split pairs	train↔test pairs
0.80–0.90	17,250	6,365
0.90–0.95	2,614	909
0.95–0.99	502	150

Table 2: Soft leakage: near-equivalent program pairs ($d \geq 1$, not byte-identical) placed in different splits by the exact gate. Distinct evaluation programs with a train near-twin: 1.84% (test and val) at ≥ 0.95 ; 5.4–5.8% at ≥ 0.90 .

Soft-leakage audit. The graded layer surfaces *near*-duplicates the exact gate cannot see. Table 2 reports pairs whose two programs sit in different splits. The benchmark-integrity figure is the fraction of evaluation programs with a near behavioural twin in *train*: at strength ≥ 0.95 , 1.84% of test and 1.84% of val programs (lower bounds—the index can miss a pair, never invent one). A disagreement-locus analysis separates genuine near-twins (disagreements confined to a single term, none in the tail) from “false friends” that agree only on a shared trivial tail; the trustworthy signal concentrates in the ≥ 0.95 band. This vindicates gating on *exact* full-signature equality rather than a similarity threshold, which would either over-merge on trivial tails or miss single-term twins.

6 The intensional layer: provable properties

Where the extensional layer reads behaviour off finitely many outputs, the intensional layer *proves* properties of the function for all inputs by sound abstract interpretation [4]. Each transfer function over-approximates the concrete operator, so a claim of “always ≥ 0 ” or “always even” holds by construction; loops are handled by Kleene fixpoints in finite lattices. We compute three properties.

Property 1 (Totality). *A program is provably total when only resource limits can stop it: it contains neither `compr` nor `pop`, and every `divi/modu` divisor is provably nonzero. This is a conservative lower bound—“unknown” does not mean partial.*

Property 2 (Sign). *A sign-domain analysis (with $x = n \geq 0$, and $a \bmod b \geq 0$ for $b > 0$) proves the OEIS keyword `nonn` (nonnegativity) for many programs.*

Property 3 (Parity). *A $\mathbb{Z}/2$ analysis proves “always even” / “always odd” (e.g. `mult` by an even factor is even).*

Soundness gates. Two independent checks validate the implementation. A static gate compares every sign/parity claim against the real signatures: 0 violations across all 104,083 programs (≈ 3.3 M value-checks). An empirical totality gate re-runs the real evaluator on 500 provably-total programs to $n = 48$ (beyond the probe window): 0 genuine errors, only resource timeouts. Table 3 gives the distribution.

Disambiguating the signature. A $\perp$ in σ is ambiguous: is the program genuinely partial, or merely slow at the probe budget? Totality resolves it. Of the 12,966 programs carrying a $\perp$, 2,345 are provably total—their $\perp$ is a *resource artefact*, not partiality—while 10,621 are not provably total, with attributed cause: 6,539 `compr` (non-terminating search), 3,023 `pop` (empty list), 1,059 `divi/modu` (possible zero divisor). The intensional layer thereby cleans the extensional signal.

provable property	count	share
total	37,996	36.5%
nonnegative (nonn)	45,251	43.5%
positive	14,907	14.3%
even	2,725	2.6%
odd	2,926	2.8%

Table 3: Provable-property distribution over 104,083 programs. Shares are lower bounds: the analysis is conservative.

7 Filtering and measuring in the OEIS loop

Both roles Table 3 enables are useful inside a synthesis loop.

Filtering. An OEIS target often carries known properties—its keywords (**nonn**, **sign**, **mult**), or properties provable from a few terms. A candidate program whose *provable* properties contradict the target’s is provably wrong and can be pruned before it is ever run: if the target is nonnegative, every provably-negative candidate is rejected soundly. Lifted into the decoder, this is a *verified semantic mask* on top of the syntactic legality mask of Section 4: constrain generation to programs provably producing outputs of the required sign/parity. Because the properties are sound, such filtering never discards a correct program.

Measuring. The same properties are theoretical measurements of a program and of a sequence. Totality distinguishes a genuinely partial sequence from a slow-to-compute total one; the sign/parity certificates are machine-checked versions of OEIS keywords, so the corpus can be scored—and OEIS annotations predicted and *verified*—at scale. More finely, the graded truth values give each program pair a calibrated similarity, and the provable properties give certain-confidence intensional features that pool with behavioural evidence by probabilistic-logic revision. The corpus becomes not merely programs and outputs but programs, outputs, and *proofs*.

8 The loop compounds: a three-seed cumulative replication

The construction above is only interesting if the loop it serves actually runs. It does. We report a prospectively registered replication of the self-learning regime of [1] using a typed-action decoder over the verified skeleton of Section 4: the network never emits program text, only *legal construction actions* over an exact state, and the checker disposes.

The protocol is the cumulative one. Each of three independently seeded worlds—seeds are random-number labels only, controlling initialisation, minibatch order, dropout, and search tie-breaking; the data pool (79,249 rows) and the 240 search tasks are shared—trains on its accumulated verified corpus, searches under a fixed budget, checks every candidate against the whole OEIS, adds newly covered sequences to its corpus, and repeats. The endpoint is cumulative coverage C_g and its increment $D_g = C_g - C_{g-1}$; no comparison against held-back external data governs continuation, since in the deployed regime no such reservoir exists.

The shared initial corpus is not chosen freely: it is the reference system’s own public bootstrap seed `so10` [1], 3,771 sequences found by *random* program search with no learning, and the very origin from which that system subsequently grew its full published corpus through many iterations

of the loop we are running here. Starting from that seed rather than from a mature corpus is deliberate, with three consequences. The origin is reproducible and learning-free, a fixed public artefact anyone can replay from. Our run is then the *same* self-learning process re-executed—not a fork of another system’s answers—so a rediscovery is a genuine held-out generalisation within our own lineage rather than a lookup. And the base stays small enough that a world’s own discoveries form a measurable share of the training signal instead of being drowned by it, which is what makes the loop’s dynamics observable at all. Coverage per world:

world	initial	after gen. 1	after gen. 2
17	3,771	3,789	3,846
29	3,771	3,841	3,891
43	3,771	3,807	3,862

At generation three we registered, before any update, a three-way comparison of how the accumulated discoveries enter training: *uniform* replay (literal pooling, under which the 194 cumulative discoveries receive their natural 0.24% of sampling mass); *balanced* replay (25% discovery mass); and balanced replay with the auxiliary predictive heads disabled. All three arms continue from the same per-world parent checkpoint under identical update counts and search budgets, and are judged solely by new whole-OEIS A-numbers beyond the pooled 3,986 already known. New A-numbers, generation three:

setting	world 17	world 29	world 43	pooled
uniform replay	48	39	72	159
balanced replay, aux. off	65	155	88	308
balanced replay, aux. on	135	155	177	467

Every setting is positive in every world: nine of nine cells extend coverage, so the loop compounds rather than saturating at this scale. Two registered contrasts separate the mechanism. Balanced replay is worth +308 pooled over uniform—at literal pooling the model’s own discoveries are numerically drowned by the base corpus, and the loop is starved of exactly the signal it exists to generate. The auxiliary heads are worth a further +159. The generation-three increment under the selected setting exceeds the generation-two increment by roughly threefold, and the setting carried into generation four was fixed by the registered selection rule rather than chosen after inspection.

Two negative results bound the claim, and we record them because they cost us the distinction. First, an ablation asking whether a world’s own discoveries beat an equal mass of *unused external certified examples* answers no, in all three worlds (162 versus 253 pooled): feedback submitted more candidates yet explored fewer distinct programs (5,540 versus 6,233), a diversity contraction rather than a fitting failure. That comparison is a legitimate ablation and a poor objective—the fresh arm is an oracle-advantaged comparator that does not exist in the deployed loop, where the corpus is whatever the loop has verified. Second, no arm in any generation solved its own assigned held-out target within budget. The evidence therefore concerns learned *discovery* guidance over the catalogue—the regime [1] operates in—and not target-conditioned synthesis, which remains open.

What the verified layer contributes here is attributability rather than yield. Because the legality mask is machine-checked, a discovery cannot be an artefact of a malformed candidate slipping past a hand-written filter; because ranking is provably recall-preserving, guidance changes search order and cannot manufacture or destroy solutions; and because every accepted program is checked against the whole corpus, the increments above are exact rather than estimated. The loop’s growth and the loop’s trustworthiness are established by different machinery, and both are needed.

9 From leakage to conjecture

Near-equivalence is dual to conjecture: what threatens a benchmark as contamination is, read the other way, a discovered near-identity. The sharpest instance is the pairing of the *fastest* and the *smallest* program for a sequence. These minimize opposite terms of Levin’s Kt complexity, $Kt(x) = \min_p |p| + \log t(p)$ [5]: program length and running time. Two quasi-independent searches, under different inductive biases, converging on the same total function is strong corroboration that the function is the intended sequence—never entailment (finitely many terms cannot pin an infinite sequence), but Bayesian evidence, and in probabilistic-logic terms the revision of two independent estimates into higher confidence. *Proving* the two programs equivalent on all n converts finite-prefix agreement into total agreement; the induction predicates such proofs require are exactly what recent conjecturing systems learn to generate [2]. Our verified layer offers to discharge them as kernel-checked theorems rather than trusted external calls. The interesting conjectures live where extensional similarity is high but *syntactic* intensional similarity is low—two unlike programs computing one sequence—and shared *provable* properties become lemmas toward the identity proof.

10 Status and roadmap

The verified DSL semantics, recognizer, big-step modalities, and the three crown obligations are machine-checked and axiom-clean. The synthesizer’s data pipeline, exact split gate, and the WM-PLN extensional and intensional layers are implemented and measured over the current corpus, with the numbers reported above; the self-learning loop of Section 8 compounds under three registered replay settings across three worlds. Near-term work: (i) bind the Lean canonical policy to the exported operator-table metadata so the ML mask and the proved canonical form share one source of truth; (ii) lift the sound Python properties to Lean theorems, closing the verified-semantic-mask loop; (iii) retain the fastest and smallest program per sequence so the within-sequence equivalence-conjecture analysis of Section 9 becomes runnable; (iv) add growth-class and monotonicity to the abstract interpreter. The unifying aim is an OEIS synthesis loop in which every accepted program is both behaviourally corroborated and provably constrained.

References

- [1] T. Gauthier and J. Urban. *Learning Program Synthesis for Integer Sequences from Scratch*. AAAI 2023; arXiv:2202.11908.
- [2] T. Gauthier and J. Urban. *Learning Conjecturing from Scratch*. arXiv:2503.01389, 2025.
- [3] OEIS Foundation Inc. *The On-Line Encyclopedia of Integer Sequences*. <https://oeis.org>.
- [4] P. Cousot and R. Cousot. *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. POPL 1977.
- [5] L. A. Levin. *Universal Sequential Search Problems*. Problems of Information Transmission, 9(3):265–266, 1973.
- [6] B. Goertzel, M. Iklé, I. Goertzel, and A. Heljakka. *Probabilistic Logic Networks*. Springer, 2009.